Title: Taming the Market: A Practical Guide to Overfitting in Decision Trees

GitHub Repository Link:
[https://github.com/mur457/decision-tree-overfitting]

1. Introduction: The Danger of Memorization

When I first started exploring machine learning, Decision Trees felt like magic. You feed them raw data, and they automatically figure out exactly what questions to ask to make accurate predictions. However, when I tried applying a standard Decision Tree to financial markets, I immediately hit a wall. My model looked like a genius when analyzing historical data, but it failed completely when trying to predict the future.

This tutorial is about why that happens. We are going to explore the concept of **overfitting**—the moment a machine learning model stops actually "learning" and just starts memorizing the data.

To demonstrate this, I have chosen to use historical stock market data from the Nifty 50 index. Financial data is inherently noisy and unpredictable. Because there is so much random fluctuation in stock prices, it serves as the absolute perfect sandbox to watch a Decision Tree overfit in real-time. By the end of this tutorial, you will understand exactly why unconstrained models fail, and how to use pruning techniques to fix them.

2. The Theory: How Trees Learn (and Fail)

To understand overfitting, we have to look under the hood at how a Decision Tree makes its rules. The algorithm's primary goal is to split data into groups that are as "pure" as possible. It measures this purity using a mathematical concept called **Entropy**.

Entropy represents the amount of chaos or uncertainty in a dataset. If a dataset has an equal number of days where the market went up and days where the market went down, the entropy is high. The formula for Entropy (H) of a dataset (S) is:

$H(S) = - \Sigma p_i * \log_2(p_i)$
(Where $p_i is the probability of a specific outcome).

The algorithm tests thousands of different splits and chooses the one that drops the entropy closest to zero.

The Problem: If you do not give the algorithm any rules, it will keep building deeper and deeper branches until every single final leaf has an entropy of exactly zero. In a noisy environment like the stock market, the tree isn't finding real economic trends; it is just creating a highly specific, convoluted rule for every single random blip that happened in the past.

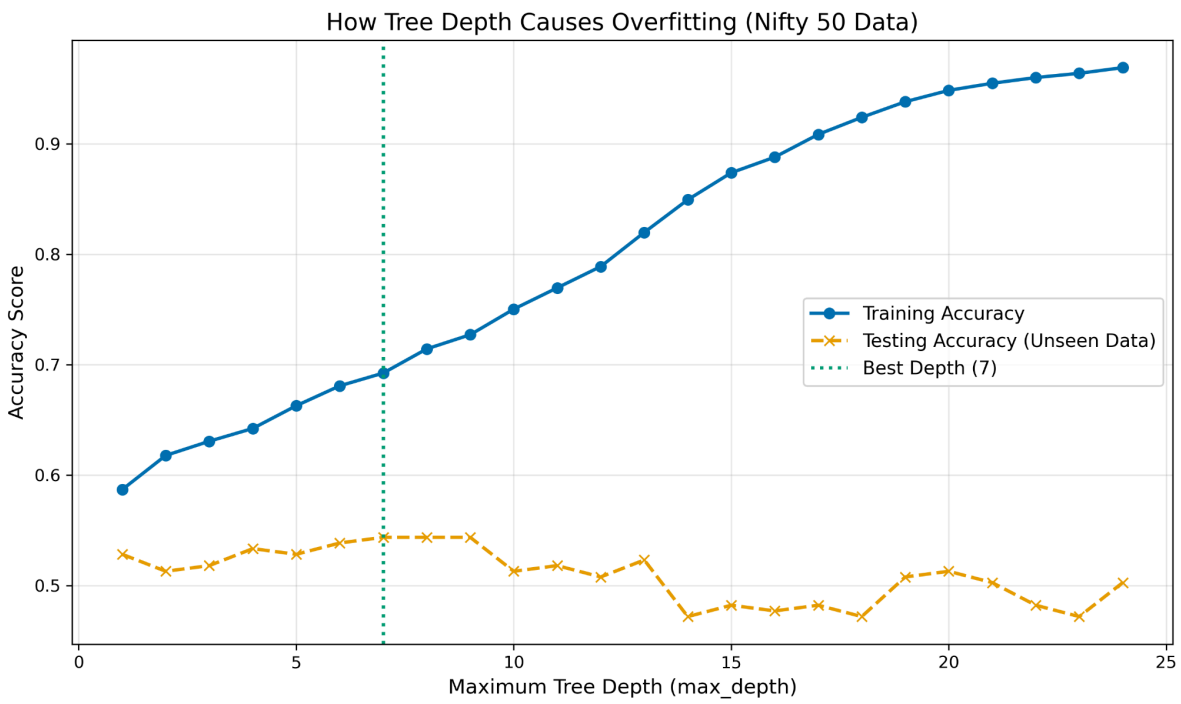
3. The Experiment and Methodology

To visualize this, I wrote a Python script to run a controlled experiment.

1. **The Dataset:** I used the `yfinance` library to download four years of daily closing prices for the Nifty 50 index.
2. **The Features:** I calculated the 5-day Simple Moving Average (SMA), the 20-day SMA, and the daily percentage return. These act as the clues for our model.
3. **The Target:** The model was tasked with predicting a binary outcome: `1` if the market closes higher the next day, and `0` if it closes lower.
4. **The Test:** I split the data, holding back 20% of it as a hidden "test" set. I then trained 24 separate Decision Trees inside a loop. The only variable I changed was the `max_depth` hyperparameter—restricting how many consecutive questions the tree was allowed to ask, from 1 all the way to 24.

4. Findings and Insights

By recording the accuracy of all 24 models on both the training data and the hidden testing data, I generated the following visualization:



Accessibility Note: When designing this plot, I intentionally chose a colorblind-friendly palette (safe for Deuteranomaly) and utilized distinct line markers (circles vs. crosses) so that the diverging trends are clearly readable to everyone, without relying on color alone.

Insight 1: The Illusion of Perfection

Look at the solid blue line (Training Accuracy). As the tree is allowed to grow deeper—past a depth of 15—the training accuracy marches straight up toward 100%. If we only looked at this metric, we would assume we had built a flawless stock market predictor.

Insight 2: The Reality Check

Now look at the dashed orange line (Testing Accuracy). As the tree gets deeper, its performance on unseen data actually gets worse, eventually hovering around 50% (no better than a coin flip). The massive, widening gap between the blue line and the orange line is the textbook visual definition of overfitting. The deep trees have memorized the historical noise, rendering them useless for future predictions.

Insight 3: Finding the Sweet Spot

By analyzing the testing data line, we can see that the model's actual predictive power peaks at a maximum depth of **7**. After depth 7, any further complexity actively harms the model.

5. Conclusion: The Solution is Pruning

The main takeaway from this experiment is that more complexity does not equal a better model. To build a robust Decision Tree, we must act as a check against the algorithm's desire to reach zero entropy.

We do this through **pruning** (hyperparameter tuning). By setting constraints like `max_depth=7` (forcing the tree to stop growing) and `min_samples_leaf=10` (refusing to create a rule unless there is a statistically significant number of days to back it up), we force the model to ignore the daily noise. It stops memorizing, and starts actually learning the broader, generalizable trends.

6. References

1.Scikit-learn Developers. (n.d.). *Decision Trees*. Scikit-learn Documentation. Retrieved from https://scikit-learn.org/stable/modules/tree.html
2.Yahoo Finance. (n.d.). *Nifty 50 (^NSEI) Historical Data*. Retrieved via the `yfinance` Python API.